Contents

Agentic Crystallization v2.2 **1**
Risk-Adaptive Verification and the Limits of Self-Regulation . . . 1
Abstract . . . 1
1. Continuity and hypotheses . . . 1
2. Model and certificate . . . 1
3. Risk-based verification . . . 2
4. Experiment . . . 2
5. Independent holdout results . . . 2
6. Failure cases and falsification . . . 3
7. Mathematical implication . . . 3
8. Reproduction . . . 4
9. Next research target . . . 4

Agentic Crystallization v2.2

Risk-Adaptive Verification and the Limits of Self-Regulation

Status: Bounded synthetic algorithmic study, not a physical phase transition, indefinite autonomy, or real Pegasus self-improvement.

Mathematics · Algorithm card · Source · Raw holdout · v2.1 paper

Abstract

Version 2.1 demonstrated a persistent synthetic procedure learner with retrospective drift alarms, but missed gradual changes and failed under misleading feedback. Version 2.2 adds an optional **predeployment audit** selected by a nominal warning-risk score and a maximum certificate age. Two whitelisted procedures compete with a safe baseline. An independent 900-world holdout across nine environmental regimes and seven policies shows a substantial advantage over periodic checking when patterns recur, but periodic checking remains superior in stable, abrupt, misleading-warning and deceptive-audit regimes. Deceptive validation observations defeat every statistical guarantee premised on honest data. This is an evidence-governed tradeoff, not a universal solution.

1. Continuity and hypotheses

- **H1:** Nonadditive procedure utility: demonstrated algebraically and in planted synthetic environments only.
- **H2:** A critical adaptive phase transition: **not demonstrated**.
- **H3:** General persistent learning-to-learn: **not demonstrated** beyond restricted synthetic transfer.
- **v2.2 hypothesis:** Risk-triggered predeployment checks may reduce unnecessary verification and stale-procedure harm relative to fixed-frequency checks. Supported only conditionally.

The full v1.0–v2.1 history, code, raw experiments, and negative results are preserved in the release.

2. Model and certificate

For each epoch t, two procedures A and B have hidden deployment gains $g_t(A), g_t(B)$ relative to a zero-gain baseline. An optional audit supplies $n = 384$ independent paired differences per procedure, each in $[-1, 1]$. For $T = 24$ prespecified epochs, two candidates and familywise risk $\alpha = 0.05$, a two-sided Hoeffding union bound yields

$$r = \sqrt{\frac{2}{n} \log\left(\frac{4T}{\alpha}\right)}.$$

A candidate is certified for the **current epoch only** when $\hat{g}_t(a) - r > 0$. This statement is conditional on honest, independent and representative audit observations. Old certificates are not valid for future epochs; every unaudited deployment is explicitly recorded as *stale*. When no procedure passes, use the baseline.

3. Risk-based verification

The observed warning Z_t describes an adjacent-epoch change. With a declared nominal hazard $h = 0.06$, warning sensitivity $s = 0.85$, and specificity $q = 0.90$, the assumed posterior is

$$p_t = \frac{hP(Z_t \mid H_t = 1)}{hP(Z_t \mid H_t = 1) + (1-h)P(Z_t \mid H_t = 0)}.$$

The controller audits if $p_t \ell m > 2nc$, with assumed harm $\ell = 0.35$, deployment size $m = 80$, observation cost $c = 0.002$, or the previous audit is at least four epochs old. If the warning is missing, audit after at most two epochs. This is a **heuristic**: the hazard and warning calibration are declared assumptions, not inferred from the test environment. No claim of optimality is made.

4. Experiment

Nine regimes: stationary, abrupt, unannounced abrupt, gradual, recurrent, missing cue, misleading cue, deceptive audit, and unlearnable. Seven policies: risk-adaptive, always audit, periodic audit every four epochs, initial-audit-only, cue-only switching, cue-triggered HOLD, and baseline. Each policy sees the same cue and pre-generated audit sample within a world and epoch; the hidden true gain is used only by the external evaluator. A 40-seed development cohort is separate from the 100-seed-per-regime holdout and 50-seed-per-regime replication. Each world has 24 epochs.

Per-epoch utility is $80 g_t(a_t) - C_{\text{audit}}$. Harmful deployment counts, HOLDs, audits and stale deployments are separately reported. Results are not from real language-model or Pegasus tasks.

5. Independent holdout results

Regime	Risk utility	Periodic utility	Always utility	Risk minus periodic, paired 95% descriptive bootstrap
Stationary	27.53	27.62	26.44	-0.09 [-0.10, -0.08]
Abrupt	27.20	27.62	26.46	-0.41 [-0.67, -0.19]
Unannounced Abrupt	26.14	27.62	26.45	-1.48 [-1.95, -1.07]
Gradual	17.49	17.42	16.92	+0.08 [-0.24, +0.43]
Recurrent	25.96	18.28	26.44	+7.68 [+7.21, +8.10]
Missing Cue	27.12	27.57	26.41	-0.45 [-0.57, -0.36]

Regime	Risk utility	Periodic utility	Always utility	Risk minus periodic, paired 95% descriptive bootstrap
Misleading Cue	24.17	27.62	26.45	-3.44 [-3.62, -3.27]
Deceptive Audit	-9.86	-9.72	-10.88	-0.15 [-0.25, -0.09]
Unlearnable	-0.47	-0.38	-1.54	-0.08 [-0.10, -0.07]

Intervals are descriptive paired bootstraps across **independent worlds**, not across correlated epochs. They are not confirmatory hypothesis tests.

In recurrent tasks, risk-adaptive verification avoids the stale periods created by a fixed four-epoch schedule. In stable tasks, the initial-audit-only policy is cheaper, and periodic verification slightly outperforms risk-adaptive verification. In abrupt tasks, periodic checks are also stronger. Thus there is no uniformly superior verification schedule.

6. Failure cases and falsification

Unlearnable tasks: No procedure is better than baseline. The audit policies appropriately hold, but waste verification cost; baseline-only has utility zero.

Misleading warning: The cue likelihood differs from the declared sensitivity/specificity. Risk-based audits become more expensive and some harmful stale deployments occur. The controller's nominal posterior is not calibrated.

Unannounced abrupt change: A missing warning cannot prevent damage from a stale certificate; periodic and always-audit policies can outperform risk-adaptive selection.

Deceptive audit: From epoch eight, the audit channel reverses the sign of true utility. The formal concentration bound may describe the reported sample distribution but **not** the deployment distribution. Risk, periodic and always-audit policies have negative utility; the zero-gain baseline remains safe. More observations of systematically false evidence cannot restore the desired guarantee.

Stale certificates: Every unaudited deployment is recorded separately. A correct past certificate is not a guarantee about future tasks, even if it happens to remain useful.

7. Mathematical implication

The practical objective is to minimize verification cost plus expected undetected harm while respecting an explicit certificate boundary:

$$\min_{\pi} \ \mathbb{E}_{\pi}[C_{\text{audit}} + C_{\text{undetected harm}} + C_{\text{missed benefit}}].$$

This is an objective, not a theorem of optimality. Its evaluation requires calibrated warning likelihoods and honest feedback. Two future environments can have identical observed histories and opposite future procedure utilities; no history-only controller can guarantee a correct decision in both.

The v2.2 controller is a **risk-priced audit heuristic**, not a discovered physical potential or a demonstrated phase transition. H2 remains untested here; H3 remains unproven generally.

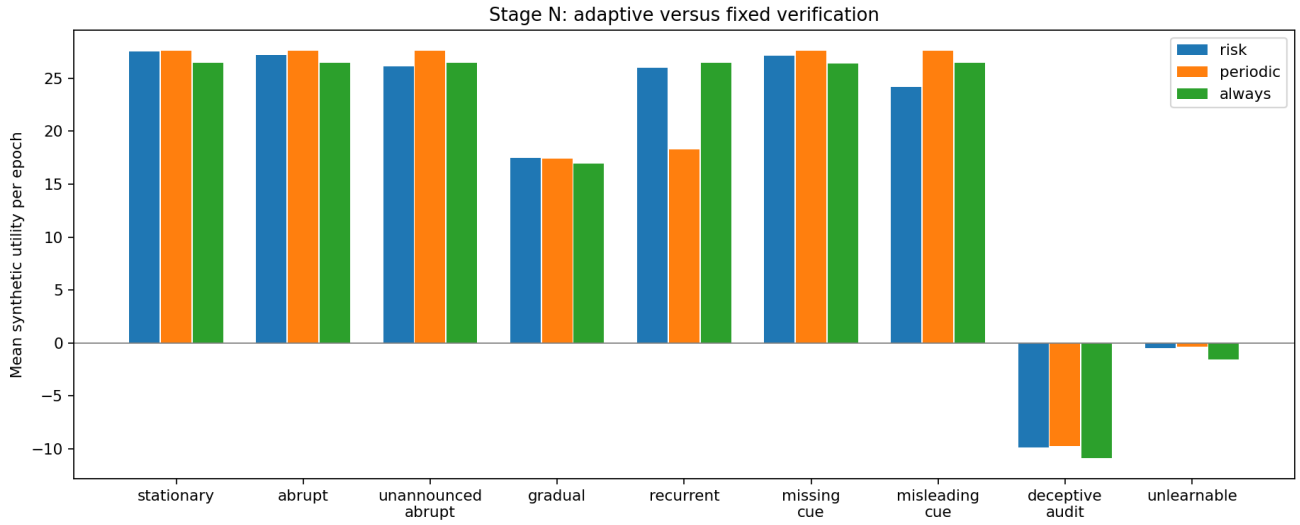


Figure 1: Stage N utility comparison

8. Reproduction

```
$env:PYTHONPATH='src'
python -m unittest discover -s tests -v
python -m agentic_crystallization.stage_n --out experiments/replay_n --start-seed 5000 --count 100
python scripts/verify_package.py .
```

The output directory must not already contain files. Replication seeds are 7000–7049. See the protocol, raw results, source, and tests.

9. Next research target

Before advancing further, validate the cue's sensitivity, specificity and change hazard using independent, labeled observations; carry uncertainty in those estimates into the audit decision; and compare against a **calibrated event-driven baseline**. Require conservative HOLD when feedback integrity cannot be established. Keep higher-order cluster formation as a separate experiment from verification scheduling.